FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate

Bài 4: Xử Lý Dữ Liệu Văn Bản

www.poly.edu.vn

AGENDA

- Dữ liệu phân loại
- Dữ liệu văn bản
- Phương pháp biểu diễn
- Thực hành

1. Dữ Liệu Phân Loại

1.1 Vấn đề dữ liệu

Trong tập dữ liệu dự đoán giá nhà, có những cột như:

- Phường: Từ Liêm, Cầu Giấy, Thanh Xuân...
- Loại nhà: Chung cư, Nhà phố, Biệt thự
- Pháp lý: Sổ hồng, Sổ đỏ, Đang chờ

->Đây là chuỗi ký tự, nhưng mô hình chỉ hiểu được số.
->Mô hình **không học được** hoặc **hiểu sai** mối quan hệ.

1.2 Định nghĩa

Dữ liệu phân loại (Categorical data) là những đặc trưng trong dữ liệu có giá trị thuộc về một tập hữu hạn các nhãn (categories), thường là dạng chữ hoặc mã, không có ý nghĩa số học.

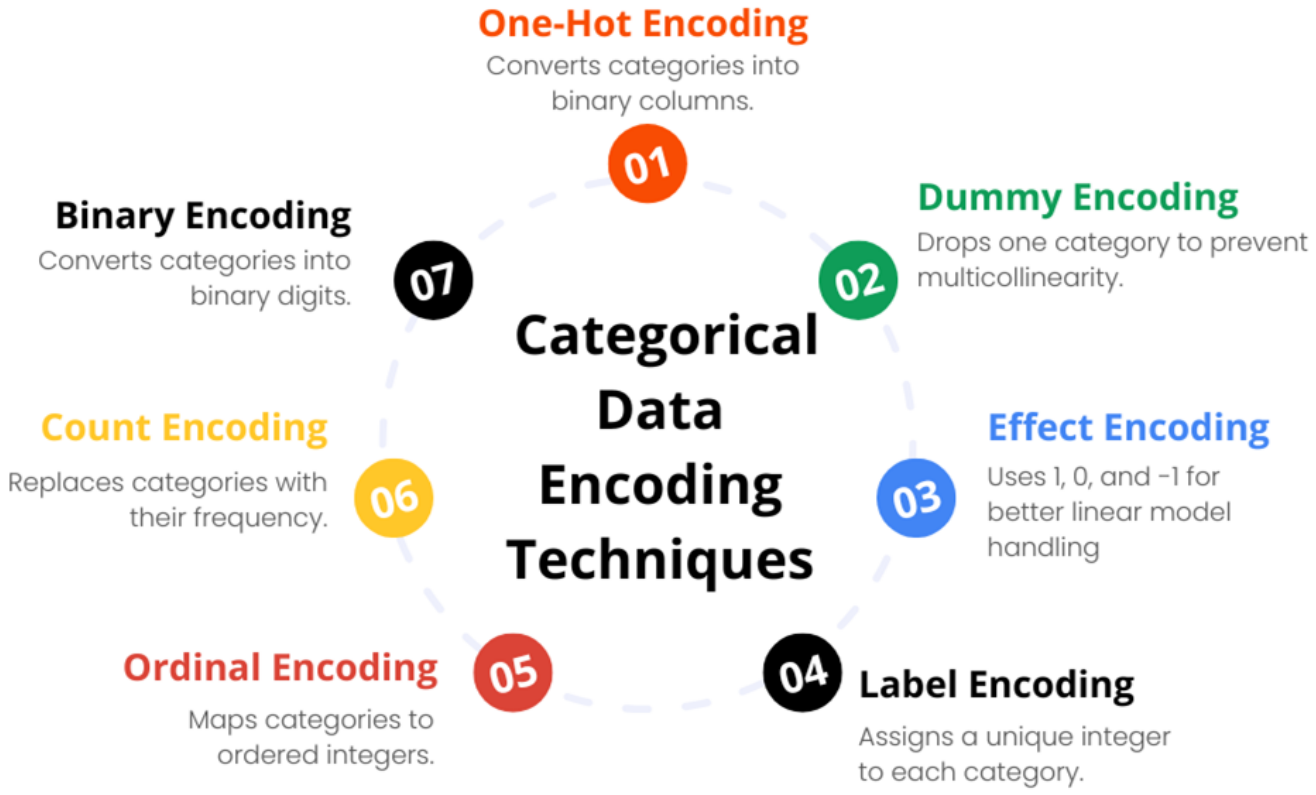
Gồm 2 loại:

- Nominal (không có thứ tự): Ví dụ: Quận, màu sắc, giới tính
- Ordinal (có thứ tự): Ví dụ: Xếp hạng chất lượng (thấp – trung bình – cao)

1.3 Giải pháp

Biến dữ liệu dạng "Văn bản" → "Số" để mô hình hiểu được.

Quá trình này gọi là: Categorical Encoding



1.3.1 Giải pháp - Label Encoding

Là phương pháp mã hóa dữ liệu phân loại bằng cách gán mỗi giá trị (label) một số nguyên duy nhất.

Ví dụ: Cột Phường có 3 giá trị

Giá trị gốc	Sau khi mã hóa
"Thanh Xuân"	0
"Từ Liêm"	1
"Cầu Giấy"	2

1.3.1 Giải pháp - Label Encoding

- Ưu điểm:

- Đơn giản, dễ triển khai, chỉ cần gán số
- Chỉ cần 1 cột số thay vì nhiều cột
- Tốc độ cao, phù hợp với dữ liệu có nhiều nhãn

- Nhược điểm:

- Tạo thứ tự giả, số gán tạo cảm giác có “thứ tự” giữa các nhãn
- Gây nhiễu cho mô hình, một số mô hình hiểu nhãn 2 > nhãn 1 > nhãn 0 là có ý nghĩa về “mức độ”

1.3.1 Giải pháp - Label Encoding

Ví dụ: Cột “Loại nhà”

Chỉ cần 1 cột số → gọn nhẹ, dễ huấn luyện

Phù hợp với mô hình tree-based (như XGBoost, LightGBM) – không bị ảnh hưởng bởi thứ tự.

Loại nhà	Mã hóa Label
Biệt thự	0
Chung cư	1
Nhà phố	2

Nếu dùng với Linear Regression, Logistic Regression, KNN…

→ Mô hình sẽ hiểu nhầm: "Nhà phố (2) > Chung cư (1) > Biệt thự (0)"

→ Khác thực tế không có thứ tự

1.3.2 Giải pháp - One-Hot Encoding

Là phương pháp mã hóa dữ liệu phân loại bằng cách tạo một cột nhị phân (0 hoặc 1) cho mỗi giá trị phân loại.

Ví dụ: Cột Phường có 3 giá trị

Giá trị gốc	Thanh_Xuan	Cau_Giay	Tu_Liem
"Thanh Xuân"	1	0	0
"Cầu Giấy"	0	1	0
"Từ Liêm"	0	0	1

1.3.2 Giải pháp - One-Hot Encoding

- Ưu điểm:

- Không tạo thứ tự giả, tránh hiểu sai mối quan hệ giữa các nhãn
- Phù hợp với nhiều mô hình: Linear Regression, Logistic Regression, Neural Networks...

- Nhược điểm:

- Tăng số chiều dữ liệu, với nhiều nhãn → số cột tăng rất nhanh (gọi là "curse of dimensionality")
- Tốn bộ nhớ & tính toán, không phù hợp với tập dữ liệu lớn có nhiều giá trị phân loại

1.3.2 Giải pháp - One-Hot Encoding

Ví dụ: Cột "Loại nhà"

→ Mô hình không "hiểu nhầm" rằng Biệt thự > Chung cư như Label Encoding có thể gây ra.

Loại nhà	Biệt thự	Nhà phố	Chung cư
Biệt thự	1	0	0
Nhà phố	0	1	0
Chung cư	0	0	1

Nếu cột có 1000 giá trị khác nhau (ví dụ: 1000 phường/địa phương)

→ Sẽ tạo 1000 cột → bùng nổ số chiều dữ liệu

→ Gây nặng nề cho mô hình

1.3 Giải pháp - So sánh

Tiêu chí	Label Encoding	One-Hot Encoding
Cách hoạt động	Gán mỗi nhãn 1 số nguyên duy nhất	Tạo 1 cột nhị phân cho mỗi nhãn
Kết quả mã hóa	Một cột số nguyên	Nhiều cột 0/1 (số cột = số nhãn)
Phù hợp mô hình	Tree-based (XGBoost, Random Forest...)	Linear, Logistic, Neural Networks

1.3 Giải pháp - So sánh

Trường hợp dữ liệu	Phương pháp nên dùng	Ghi chú
Dữ liệu có ít nhãn (dưới 10–20)	One-Hot Encoding	Tránh tạo thứ tự giả, dễ hiểu
Dữ liệu có nhiều nhãn (50+)	Label Encoding	Tránh tăng số chiều, tiết kiệm bộ nhớ
Dữ liệu có thứ tự thực sự (Low < Medium < High)	Label Encoding	Thứ tự có thể mang ý nghĩa cho mô hình
Dữ liệu đưa vào mô hình tuyến tính	One-Hot Encoding	Vì các mô hình tuyến tính nhạy với thứ tự giả
Dữ liệu đưa vào mô hình cây quyết định	Label Encoding	Mô hình cây không bị ảnh hưởng bởi thứ tự giả

2. Dữ Liệu Văn Bản

2.1 Vấn đề dữ liệu

Trong thực tế, nhiều thông tin quan trọng không chỉ nằm ở các nhãn cố định, mà còn nằm trong phần mô tả văn bản tự do

Ví dụ: “Nhà đẹp, gần trường học, diện tích rộng, khu an ninh…”

-> Đây là dữ liệu văn bản, chứa nhiều chi tiết và ý nghĩa phong phú.
-> Giúp hiểu rõ hơn về đặc điểm sản phẩm, cảm nhận khách hàng, các tiện ích xung quanh…

2.2 Định nghĩa

Dữ liệu văn bản (Text Data) là dữ liệu dưới dạng chuỗi ký tự tự do, bao gồm các từ, câu, đoạn văn, hoặc thậm chí toàn bộ tài liệu.

- Chứa thông tin ngữ nghĩa (về đối tượng, sự kiện, cảm xúc, mô tả, v.v.)

- Phổ biến trong các bài toán NLP (Natural Language Processing) như phân tích cảm xúc, nhận diện chủ đề, hệ thống đề xuất, v.v.

2.2.1 Định nghĩa - Ví dụ

Ví dụ mô tả:

"Nhà đẹp, gần trường học, diện tích rộng, khu an ninh tốt, giao thông thuận tiện."

Dữ liệu này chứa thông tin quan trọng:

- Chất lượng nhà: đẹp, rộng
- Tiện ích xung quanh: gần trường học, khu an ninh
- Giao thông: thuận tiện

2.2.2 Định nghĩa - Thách thức

Thách thức:

- Không phải tất cả từ ngữ đều có giá trị tương đương

→ Cần hiểu ngữ nghĩa, ngữ cảnh sử dụng từ

- Văn bản có thể không có cấu trúc cố định

→ Các câu có thể được viết theo nhiều cách khác nhau

2.3 Các bước tiền xử lý

Câu mẫu:

"Sản phẩm này rất tốt, chất lượng tuyệt vời và giao hàng nhanh chóng."

2.3 Các bước tiền xử lý

2.3.1 Phân tách văn bản (Tokenization)

Phân tách câu thành các từ (tokens) để mô hình có thể xử lý từng phần tử của văn bản một cách độc lập.

Việc này giúp mô hình hiểu được mối quan hệ giữa các thành phần của câu.

2.3 Các bước tiền xử lý

2.3.1 Phân tách văn bản (Tokenization)

"Sản phẩm này rất tốt, chất lượng tuyệt vời và giao hàng nhanh chóng."

["Sản", "phẩm", "này", "rất", "tốt", "chất", "lượng", "tuyệt", "vời", "và", "giao", "hàng", "nhanh", "chóng"]

2.3 Các bước tiền xử lý

2.3.1 Phân tách văn bản (Tokenization)

Nếu không tách câu thành các token, mô hình sẽ không thể phân tích ý nghĩa của từng từ riêng lẻ.

Việc phân tách giúp mô hình nhận diện các từ quan trọng như "tốt", "chất lượng", "nhanh chóng" và "giao hàng".

2.3 Các bước tiền xử lý
2.3.2 Chuyển thành chữ thường (Lowercase)

Chữ hoa và chữ thường không có sự khác biệt về nghĩa trong văn bản tiếng Việt.

Việc chuyển tất cả các từ về dạng chữ thường giúp mô hình

- Không phân biệt sai giữa các từ giống nhau.
- Không phải học lại những từ tương tự với chữ hoa và chữ thường khác nhau.

Ví dụ, từ "Sản" và "sản" sẽ được coi là một từ duy nhất, giúp giảm sự phức tạp.

2.3 Các bước tiền xử lý

2.3.3 Bỏ dấu (Normalization)

Trong tiếng Việt, dấu thanh có thể thay đổi nghĩa của từ.

Tuy nhiên, trong một số trường hợp, bỏ dấu vẫn có thể giúp chuẩn hóa văn bản mà không làm mất đi ngữ nghĩa chính.

Đặc biệt là khi xử lý văn bản để phân tích cảm xúc hoặc các đặc trưng chung của văn bản.

2.3 Các bước tiền xử lý

2.3.3 Bỏ dấu (Normalization)

Ví dụ: "tuyệt vời" -> "tuyet voi"

Việc bỏ dấu giúp giảm số lượng biến thể của từ, giúp mô hình dễ dàng phân tích và nhận diện các từ mà không phải xử lý nhiều biến thể

2.3 Các bước tiền xử lý

2.3.4 Loại bỏ từ dừng (Stopword)

Stop words là những từ xuất hiện quá thường xuyên trong ngữ cảnh mà không mang lại giá trị thông tin.

Ví dụ như "và", "là", "của", "the" trong tiếng Anh hay "và", "này" trong tiếng Việt. Chúng không giúp ích cho việc phân loại hoặc phân tích cảm xúc trong văn bản.

2.3 Các bước tiền xử lý
2.3.4 Loại bỏ từ dừng (Stopword)

"Sản phẩm này rất tốt, chất lượng tuyệt vời và giao hàng nhanh chóng."

["sản", "phẩm", "tốt", "chất", "lượng", "tuyệt", "vời", "giao", "hàng", "nhanh", "chóng"]

2.3 Các bước tiền xử lý

2.3.4 Loại bỏ từ dừng (Stopword)

Loại bỏ các từ này giúp giảm sự phức tạp trong dữ liệu, tiết kiệm tài nguyên tính toán và giúp mô hình tập trung vào các từ có ý nghĩa hơn trong việc phân tích ngữ nghĩa, ví dụ như "tốt", "chất lượng", "nhanh chóng".

2.4 Vector hóa văn bản (Vectorize)

Tại sao cần vector hóa?

- Máy tính không hiểu văn bản thô.
- Cần chuyển từ dạng chữ → dạng số (vector).
- Là bước trung gian giữa xử lý ngôn ngữ và mô hình học máy.

Phương pháp	Đặc điểm chính
Bag of Words (BoW)	Đếm số lần xuất hiện từ, đơn giản, không có ngữ cảnh
TF-IDF	Tính trọng số từ theo tần suất & mức độ quan trọng
Word2Vec	Biểu diễn ngữ nghĩa, từ giống nhau có vector gần nhau
BERT, FastText	Mô hình hiện đại, hiểu sâu ngữ cảnh

2.4 Vector hóa văn bản (Vectorize)

2.4.1 Bag of Words

Đặc điểm:

- Đếm tần suất từ trong văn bản.
- Mỗi văn bản → vector với số lần xuất hiện của từ.
- Không xét ngữ cảnh, chỉ quan tâm từ gì xuất hiện bao nhiêu lần

Ví dụ: Hệ thống lọc email rác, email có nội dung khác nhau nhưng thường chứa những từ phổ biến như “miễn phí”, “trúng thưởng”, “mua ngay”.

2.4 Vector hóa văn bản (Vectorize)

2.4.1 Bag of Words

Dữ liệu minh họa

Email A: “Bạn đã trúng thưởng!”
Email B: “Mua ngay với giá ưu đãi!”

Từ vựng	Email A	Email B
bạn	1	0
đã	1	0
trúng	1	0
thưởng	1	0
mua	0	1
ngay	0	1
ưu đãi	0	1

2.4 Vector hóa văn bản (Vectorize)

2.4.2 Term Frequency – Inverse Document Frequency

Viết tắt: TF-IDF

Đặc điểm:

- TF: đo tần suất từ trong văn bản
- IDF: giảm trọng số từ xuất hiện quá nhiều (như "sản phẩm", "tôi")
- → Giữ lại các từ đặc trưng & quan trọng

Ví dụ: Hệ thống phân tích đánh giá sản phẩm xác định từ nào quan trọng trong nhận xét, ví dụ như "giao hàng nhanh" hay "chất lượng kém".

2.4 Vector hóa văn bản (Vectorize)

2.4.2 Term Frequency – Inverse Document Frequency

Dữ liệu minh họa

Đánh giá 1: "Sản phẩm tốt, giao hàng nhanh."

Đánh giá 2: "Tôi hài lòng với chất lượng."

Từ "sản phẩm", "tôi" xuất hiện nhiều → IDF thấp

Từ "giao", "nhanh", "tốt" → xuất hiện ít → IDF cao → Trọng số cao hơn

2.4 Vector hóa văn bản (Vectorize)

2.4.2 Term Frequency – Inverse Document Frequency

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF-IDF(t,d) = TF(t,d) * IDF(t)$$

2.4 Vector hóa văn bản (Vectorize)
2.4.3 Word2Vec

Tư tưởng chính:
"Các từ xuất hiện trong cùng ngữ cảnh thường có ý nghĩa tương tự."

Đặc điểm:
- Học vector biểu diễn từ dựa trên ngữ cảnh
- Từ xuất hiện trong ngữ cảnh giống nhau → vector gần nhau
- Hiểu ngữ nghĩa & mối liên hệ giữa các từ

2.4 Vector hóa văn bản (Vectorize)
2.4.3 Word2Vec

Ví dụ: Hệ thống tự động đề xuất sản phẩm dựa trên mô tả

Trong không gian vector: “ấm” gần với “lạnh”, “mùa đông”, “áo khoác”

-›. Nếu họ nói “muốn mua áo len ấm”, hệ thống hiểu rằng “áo khoác mùa đông” có thể là đề xuất hợp lý.

2.4 Vector hóa văn bản (Vectorize)

2.4.3 Word2Vec

Ví dụ: Văn bản: "Tôi học Word2Vec hôm nay"

→ Ngữ cảnh của “học”: [“Tôi”, “Word2Vec”, “hôm”, “nay”]

→ Mục tiêu: đoán được “học” từ các từ xung quanh.

2.4 Vector hóa văn bản (Vectorize)

2.4.3 Word2Vec - Trò chơi đoán từ

CBOW (Continuous Bag of Words)

- Đầu vào: các từ xung quanh
- Đầu ra: dự đoán từ trung tâm

Ví dụ: “Tôi ____ Word2Vec hôm nay”

→ Input: [“Tôi”, “Word2Vec”, “hôm”, “nay”]

→ Output: “học”

2.4 Vector hóa văn bản (Vectorize)

2.4.3 Word2Vec - Trò chơi đoán từ

Skip-Gram

- Đầu vào: từ trung tâm
- Đầu ra: dự đoán các từ xung quanh

Ví dụ: “Tôi ____ Word2Vec hôm nay”

-> Input: “học”

→ Output: [“Tôi”, “Word2Vec”, “hôm”, “nay”]

3. Thực Hành

3.1 Nạp và chuẩn bị dữ liệu

```
import pandas as pd
df = pd.read_csv("housing_data.csv")
df.head()
```

	Area	Price	Region	HouseType	Description
0	65	120000	Region_A	Apartment	Căn hộ đẹp gần trung tâm, đầy đủ nội thất.
1	80	250000	Region_B	Villa	Biệt thự rộng rãi, khu vực yên tĩnh, gần công ...
2	45	90000	Region_C	Townhouse	Nhà phố nhỏ nhưng tiện nghi, gần chợ và trường...
3	120	350000	Region_A	Villa	Nhà biệt thự sang trọng, có gara và sân vườn.
4	55	100000	Region_B	Apartment	Căn hộ giá rẻ, gần siêu thị và bến xe.

3.2 Xử lý dữ liệu Category

Label Encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["HouseType_LE"] = le.fit_transform(df["HouseType"])

print("Mapping Label Encoding:")
for label, encoded in zip(le.classes_, range(len(le.classes_))):
    print(f"  {label:10} → {encoded}")

df[["HouseType", "HouseType_LE"]].head(10)
```

3.2 Xử lý dữ liệu Category

One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False)
region_encoded = ohe.fit_transform(df[["Region"]])

region_ohe_sklearn = pd.DataFrame(region_encoded, columns=ohe.get_feature_names_out())
region_ohe_sklearn.head()
```

3.3 Xử lý dữ liệu văn bản

Làm sạch

```python
import re
import unicodedata

def remove_accents(text):
    text = unicodedata.normalize('NFD', text)
    text = text.encode('ascii', 'ignore').decode('utf-8')
    return text

stopwords = ["và", "gần", "có", "nhưng", "với", "nhiều", "rộng", "mini", "rẻ"]

def preprocess_text(text):
    text = text.lower()
    text = remove_accents(text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    # loại stop words
    text = " ".join([w for w in text.split() if w not in stopwords])
    # loại từ quá ngắn
    text = " ".join([w for w in text.split() if len(w) > 2])
    return text

df["Description_clean"] = df["Description"].apply(preprocess_text)
df[["Description", "Description_clean"]].head(10)
```

3.3 Xử lý dữ liệu văn bản

TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=15)
tfidf_matrix = tfidf.fit_transform(df["Description_clean"])

tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf.get_feature_names_out())
tfidf_df.head()
```

3.3 Xử lý dữ liệu văn bản

Word2Vec

```
!pip install gensim
from gensim.models import Word2Vec

sentences = [row.split() for row in df["Description_clean"]]
w2v_model = Word2Vec(sentences, vector_size=50, window=3, min_count=1, workers=4)

print("Model Word2Vec đã train xong!")
```

3.3 Xử lý dữ liệu văn bản

Word2Vec

```
try:
    print("Các từ gần 'nha':")
    print(w2v_model.wv.most_similar("nha"))
except KeyError:
    print("Từ 'nha' không nằm trong vocabulary!")
```

FPT Education

FPT POLYTECHNIC

THANK YOU